

Module-03**RBT Level-L1, L2****Image Processing Techniques:**

Image segmentation techniques, Descriptors and matching, Feature matching, epipolar geometry, stereo vision, Image formation and camera models: pin hole geometry, orthographic vs. perspective projection, camera calibration including intrinsic and extrinsic parameters.

Image segmentation techniques

Image segmentation is a computer vision technique that partitions an image into multiple meaningful regions or segments to simplify its representation and make analysis easier.

Types of Image Segmentation

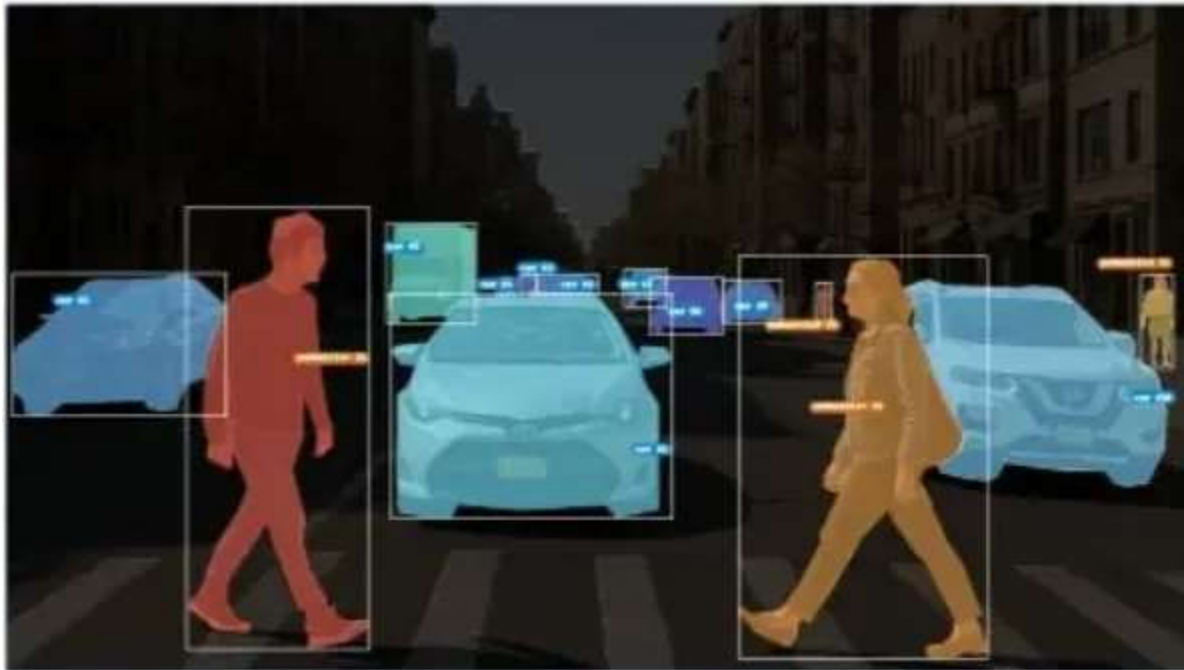
- **Semantic Segmentation**
- **Instance segmentation**
- **Panoptic Segmentation**

Semantic Segmentation: It involves assigning a class label to every pixel in an image based on shared characteristics such as colour, texture and shape.

This method treats all pixels belonging to the same class as identical, without distinguishing between individual objects.



Instance segmentation: It extends semantic segmentation by not only labelling colour of each pixel but also distinguishing between individual objects of the same class. This approach identifies each object of the same class as a unique instance.



Panoptic Segmentation: It combines both semantic and instance segmentation techniques providing a complete image analysis.

It assigns a class label to every pixel and also detects individual objects. This combined approach helps us to understand both broad categories and detailed object boundaries simultaneously.



Difference between Image Segmentation, Object Detection and Image Classification

Aspect	Image Classification	Object Detection	Image Segmentation
Purpose	Assign a label or category to the whole image	Identifies and locates multiple objects	Divide the image into meaningful regions
Output	Single label or category	Bounding boxes around detected objects	Pixel wise segmentation masks
Focus	High level classification of the entire image	Detection of objects with localization	Detailed segmentation of objects and background
Complexity	Simpler and faster	Moderate complexity	More complex and computationally intensive

Descriptors and matching in image processing

Descriptors and matching are used to find corresponding features between different images.

A **descriptor** is a compact numerical representation of a key point's local neighborhood, encoding its appearance, texture, and shape.

Matching is the process of comparing these descriptors to establish correspondences, enabling tasks like object recognition, image stitching, and 3D reconstruction

The overall process can be broken down into three steps:

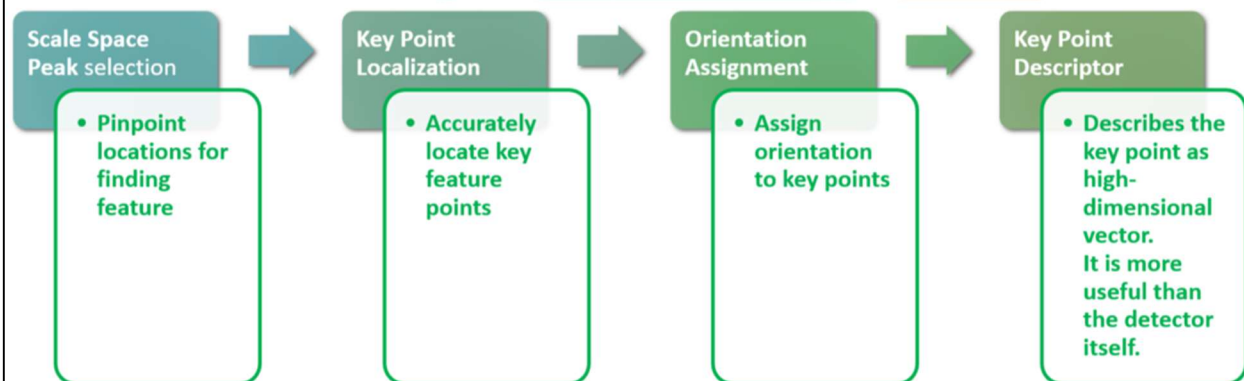
- **Feature detection:** Identify distinctive points of interest, such as corners, edges, or blobs.
- **Feature description:** For each detected feature, compute a descriptor that represents its local characteristics in a way that is robust to changes in scale, rotation, and illumination.
- **Feature matching:** Compare the descriptors from different images to find the best matches.

Common image descriptors

Scale-Invariant Feature Transform (SIFT)

Method: Creates a 128-dimensional vector by capturing gradient magnitudes and orientations in a region around a key point. It first builds a scale space using the Difference of Gaussians (DoG) to detect key points that are stable at different scales.

Properties: Invariant to scale, rotation, and to some extent, illumination changes and noise. It is computationally demanding but highly robust.



D Y PATIL
INTERNATIONAL
UNIVERSITY
AKURDI PUNE

```
# Important NOTE: Use opencv
import cv2

# Loading the image
img = cv2.imread('geeks.jpg')

# Converting image to grayscale
gray= cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

# Applying SIFT detector
sift = cv.SIFT_create()
kp = sift.detect(gray, None)

# Marking the keypoint on the image using circles
img=cv2.drawKeypoints(gray ,
                      kp ,
                      img ,
                      flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

cv2.imwrite('image-with-keypoints.jpg', img)
```

Output



The image on left is the original, the image on right shows the various highlighted interest points on the image

Speeded-Up Robust Features (SURF)

Method: A faster, more efficient alternative to SIFT that uses integral images for rapid computation of Haar wavelet responses. It creates a 64-dimensional descriptor vector.

Properties: Robust to scale, rotation, and illumination changes, with a significantly faster processing speed compared to SIFT.

Oriented FAST and Rotated BRIEF (ORB)

Method: Combines the high-speed FAST (Features from Accelerated Segment Test) keypoint detector with the BRIEF (Binary Robust Independent Elementary Features) descriptor. It adds rotation invariance by computing the intensity centroid of the feature's patch.

Properties: Extremely fast, efficient to compute and store, and robust to rotation. ORB's descriptor is a binary string, making it very suitable for real-time applications.

```
import numpy as np
import cv2
from matplotlib import pyplot as plt

img = cv2.imread('simple.jpg',0)

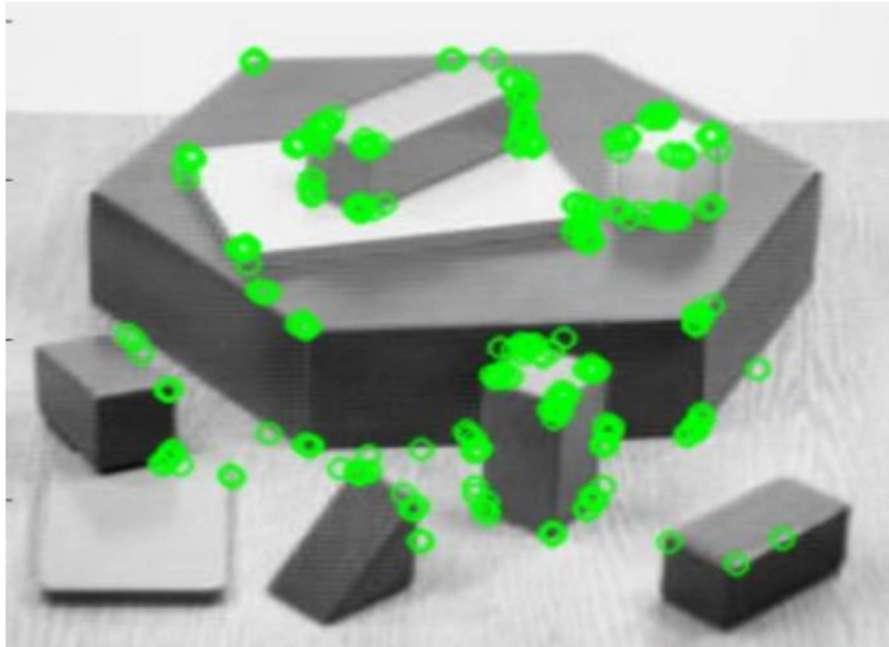
# Initiate ORB detector
orb = cv2.ORB_create()

# find the keypoints with ORB
kp = orb.detect(img,None)

# compute the descriptors with ORB
kp, des = orb.compute(img, kp)

# draw only keypoints location,not size and orientation
img2 = cv2.drawKeypoints(img, kp, None, color=(0,255,0), flags=0)
plt.imshow(img2), plt.show()
```

Output:



Local Binary Pattern (LBP)

Method: A texture descriptor that represents a neighborhood around a pixel by comparing its intensity value with that of its neighbors. This results in a binary code that can be used to form a histogram.

Properties: Computationally simple and powerful for texture classification. Basic LBP is not scale or rotation-invariant, though variants exist to address this.

Histogram of Oriented Gradients (HOG)

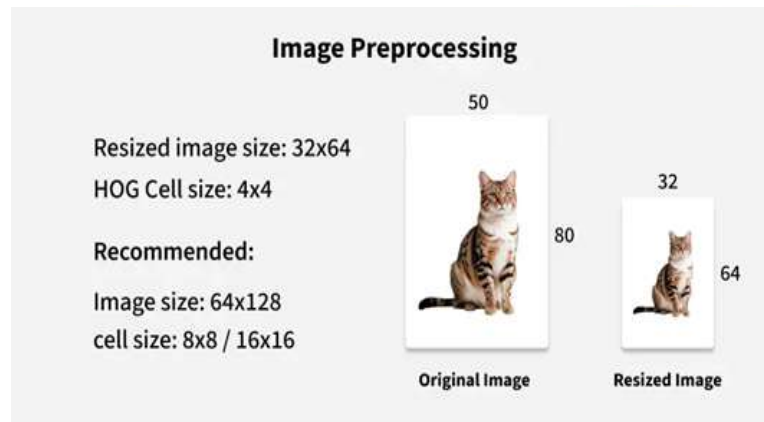
Method: Describes the appearance and shape of objects by their local intensity gradient distribution. It divides an image into small cells and aggregates histograms of gradient orientations from each cell.



1. Preprocessing

Convert the image to grayscale .

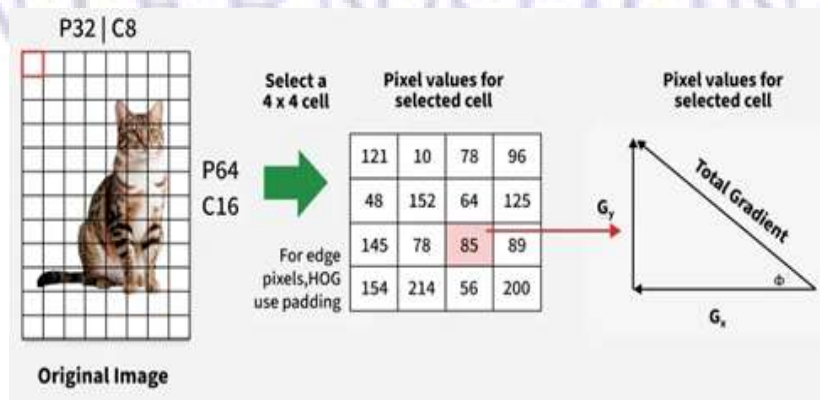
Resize the image to a standard size. As an initial step, we preprocess the image by resizing it to a standardized size of 32 pixels in width and 64 pixels in height. This ensures consistency across all input images, making further analysis and feature extraction more reliable.



Compute Gradients

- Calculate the gradient of the image in both the X and Y directions using operators like Sobel.
- For each pixel, compute:

- Gradient magnitude:** $\text{Magnitude} = \sqrt{G_x^2 + G_y^2}$
- Gradient orientation:** $\text{Orientation} = \arctan2(G_y, G_x)$



- Begin by dividing the image into small cells; in this example, the image is split into 4x4 pixel cells, resulting in 8 cells along the width (C8) and 16 along the height (C16).

- For a given cell (e.g., a 4x4 pixel block), examine the pixel value matrix within that cell.
- Value of Magnitude and direction

Gradient in x dir. (Gx)	Gradient in y dir. (Gy)
= 89-78	= 64-56
= 11	= 8
Total Gradient Magnitude	Orientation (Φ)
= $\sqrt{(G_x)^2 + (G_y)^2}$	= $\text{atan}(G_y / G_x)$
= $\sqrt{(11)^2 + (8)^2}$	= 36 deg.
= 13.6	

- Compute the gradient in the x direction (G_x) for a pixel as the difference between the pixel value to its right and the pixel value to its left (e.g., $89 - 78 = 11$).
- Compute the gradient in the y direction (G_y) as the difference between the pixel value above and the pixel value below (e.g., $64 - 56 = 8$).
- Calculate the gradient magnitude for the pixel using the Pythagorean theorem: $\sqrt{G_x^2 + G_y^2}$ (e.g. $11^2 + 8^2 = 13.6$).
- Determine the gradient orientation as the inverse tangent of G_y divided by G_x (e.g. $\arctan(8/11) = 36^\circ$).
- Repeat this process for all pixels within each 4x4 cell to obtain gradients and orientations for the entire cell.
- For edge pixels, use padding techniques to handle boundary conditions.

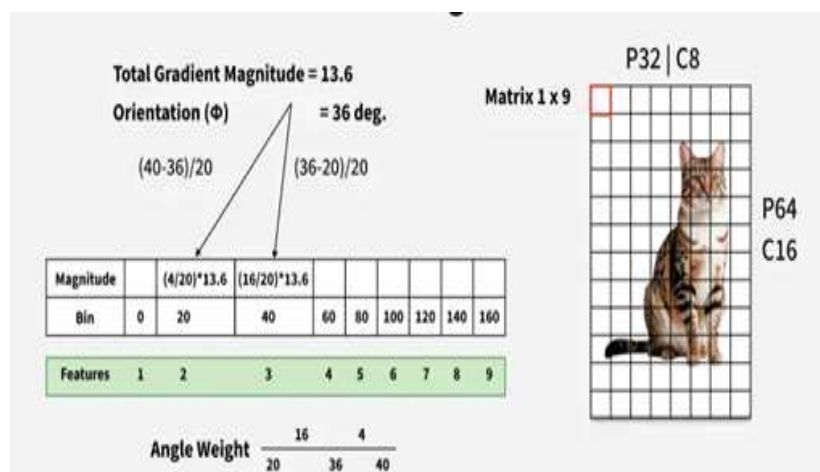
3. Divide Image into Cells

- Split the image into small, connected regions called cells (e.g., 8x8 pixels).
- Use padding to ensure that the cells at the edges of the image are at least half contained within the image, allowing the entire image area to be covered even if its dimensions are not exact multiples of the cell size.

4. Create Orientation Histograms

For each pixel, calculate the gradient magnitude (e.g., 13.6) and orientation (e.g., 36 degrees).

Orientation values range from 0 to 180 degrees.

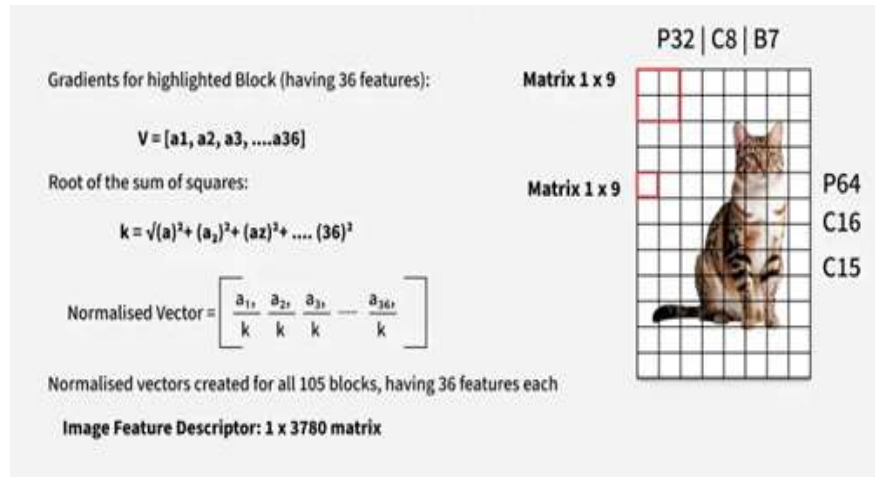


- Divide the 0–180 degree range into 9 histogram bins, each spanning 20 degrees.
- Assign each pixel's gradient magnitude to the two nearest bins, weighted by how close the orientation is to each bin center.
- For example, if a pixel's orientation is 36 degrees (closer to the 40-degree bin), most of its magnitude goes to the 40-degree bin (weight: 16/20), and the rest to the 20-degree bin (weight: 4/20).
- Repeat this process for all pixels in the cell, accumulating weighted magnitudes in each bin.
- The result for each cell is a 1x9 feature vector representing the distribution of edge directions.
- HOG computes such histograms for every cell in the image.

5. Normalize Across Blocks

- Localized image gradients are sensitive to lighting variations; normalization helps reduce this effect.
- Group neighboring cells into blocks (e.g., 4 cells per block), creating overlapping regions as the block moves one cell at a time across the image.
- For each cell, HOG creates a 1x9 feature vector (histogram of gradient orientations).

- For each block, concatenate the feature vectors of its four cells to form a 1x36 block feature vector.
- Compute the normalization factor k as the root sum of squares of all 36 block features.



- Divide each feature in the block by k to obtain the normalized block vector.
- Repeat this process for all blocks (e.g., 105 blocks when sliding across the image).
- Concatenate the normalized vectors from all blocks horizontally to form the final image descriptor (e.g., 1x3780 dimensional vector for the example case).
- This high-dimensional descriptor represents the image's edge and texture structure for use in machine learning tasks.
- Due to the large number of features, dimensionality reduction techniques like Principal Component Analysis (PCA) are often used to reduce computational complexity and prevent overfitting.

6. Concatenate Features

The final HOG descriptor is the concatenation of all normalized histograms from all blocks, forming a single feature vector.

HOG Feature Extraction: Python Example

```
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import data, exposure

image = data.astronaut()

fd, hog_image = hog(
    image,
    orientations=8,
    pixels_per_cell=(16, 16),
    cells_per_block=(1, 1),
    visualize=True,
    channel_axis=-1,
)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 4), sharex=True,
sharey=True)

ax1.axis('off')
ax1.imshow(image, cmap=plt.cm.gray)
ax1.set_title('Input image')

# Rescale histogram for better display
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=
(0, 10))

ax2.axis('off')
ax2.imshow(hog_image_rescaled, cmap=plt.cm.gray)
ax2.set_title('Histogram of Oriented Gradients')
plt.show()
```

Output

Input image



Histogram of Oriented Gradients



Output Image

Feature matching techniques

Feature matching involves comparing feature descriptors to find corresponding features across different images.

Keypoint Matching

Keypoint matching is the process of finding corresponding keypoints between different images by comparing their descriptors. The goal is to identify pairs of keypoints that represent the same physical point in the scene.

Brute-Force (BF) Matcher

The brute-force matcher compares each descriptor from one image with every descriptor from another image to find the best matches based on a chosen distance metric (e.g., Euclidean distance).

Advantages: Simple and straightforward, provides accurate matches.

Disadvantages: Computationally expensive, especially for large datasets.

K-Nearest Neighbors (KNN) Matching

KNN matching finds the k closest descriptors for each keypoint based on a distance metric. Typically, the ratio test is applied to select the best match from the k neighbors.

Advantages: More efficient than brute-force matching, allows for flexibility in selecting the best match.

Disadvantages: Still computationally intensive for very large datasets.

Robust matching with RANSAC

RANSAC is an iterative method used to estimate parameters of a mathematical model from a set of observed data that contains outliers. In feature matching, RANSAC is used to find a robust set of matches by repeatedly selecting random subsets of matches and computing a transformation that aligns the images. The transformation with the highest number of inliers is chosen.

Advantages: Robust to outliers and noise, improves the accuracy of feature matching.

Disadvantages: Computationally intensive, requires careful parameter tuning.

Epipolar geometry in computer vision

Epipolar geometry in computer vision describes the relationship between two images of the same 3D scene taken from different viewpoints, establishing constraints that reduce the search for corresponding points between the images.

Key Concepts

Epipole (e): The point where the line connecting the two camera centers (the baseline) intersects the image plane of the other camera. For every pair of cameras, there is a left epipole and a right epipole.

Epipolar Plane: A plane in 3D space defined by a 3D scene point and the optical centers of both cameras. It represents the "ray of sight" from each camera to that 3D point.

Epipolar Line: The intersection of the epipolar plane with an image plane. For any given point in one image, its corresponding point in the other image must lie on its respective epipolar line

```
import numpy as np
import cv2
from matplotlib import pyplot as plt

# Load the Left and right images
# in gray scale
imgLeft = cv2.imread('image_l.png',
                     0)
imgRight = cv2.imread('image_r.png',
                     0)

# Detect the SIFT key points and
# compute the descriptors for the
# two images
sift = cv2.xfeatures2d.SIFT_create()
keyPointsLeft, descriptorsLeft = sift.detectAndCompute(imgLeft,
                                                         None)

keyPointsRight, descriptorsRight = sift.detectAndCompute(imgRight,
                                                         None)

# Create FLANN matcher object
FLANN_INDEX_KDTREE = 0
indexParams = dict(algorithm=FLANN_INDEX_KDTREE,
                   trees=5)
searchParams = dict(checks=50)
flann = cv2.FlannBasedMatcher(indexParams,
                              searchParams)
```

```
# Apply ratio test
goodMatches = []
ptsLeft = []
ptsRight = []

for m, n in matches:

    if m.distance < 0.8 * n.distance:

        goodMatches.append([m])
        ptsLeft.append(keyPointsLeft[m.trainIdx].pt)
        ptsRight.append(keyPointsRight[n.trainIdx].pt)
```

Image Left



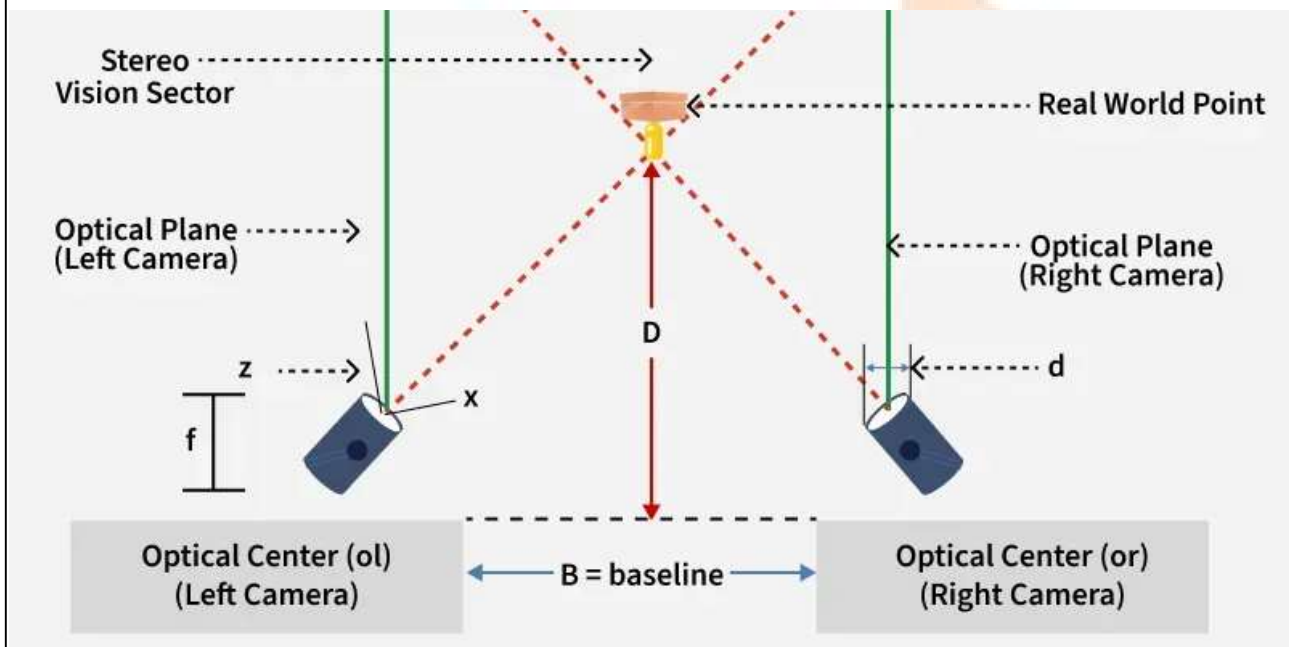
Image Right



Stereo Vision and Depth Estimation

Stereovision (or stereo vision) is a technique in computer vision that uses two or more cameras placed at different viewpoints to simulate human binocular vision. It allows the perception of depth by identifying corresponding points in the images taken from each camera

Working of Stereo Vision



Capture Images: Use two synchronized cameras with a fixed baseline to capture left and right images of a scene.

Camera Calibration: Estimate intrinsic parameters (focal length, principal point) and extrinsic parameters (rotation and translation between cameras).

Image Rectification: Warp both images so that epipolar lines are aligned horizontally. This simplifies correspondence search to one dimension (along scanlines).

Stereo Matching / Disparity Estimation: Find corresponding pixels in both images using Block Matching, Semi-Global Matching (SGM), Graph Cuts and Deep Learning.

Disparity Map Construction: Generate a disparity image where each pixel stores the disparity value (in pixels).

Depth Estimation (Triangulation): Compute depth using the formula:
 $Z=(f.B)/D$

Depth Map and Visualization: Generate a 2D depth map or point cloud to represent 3D structure.

Mathematical Representation

To calculate disparity and depth:

Let:

- f = focal length (in pixels)
- B = baseline (distance between cameras)
- d = disparity (difference in x-coordinates between matching pixels)

Depth (Z) is calculated as:

$$Z=(f.B)/D$$

Disparity:

$$D=X_{\text{left}}-X_{\text{right}}$$

Advantages of Stereovision

- No need for active sensors.
- Relatively low-cost with commercial stereo cameras.
- Useful in both indoor and outdoor environments.
- Good depth resolution for nearby objects.
- Can be used in real-time systems.

Applications of Stereovision

- Autonomous Vehicles for obstacle detection and navigation.
- Robotics for grasping and interaction with environments.
- Medical Imaging.
- Industrial Inspection

Image formation and camera models: pinhole geometry

The pinhole geometry in a camera model explains how 3D points in a scene are mapped to 2D points on an image plane.

How it Works

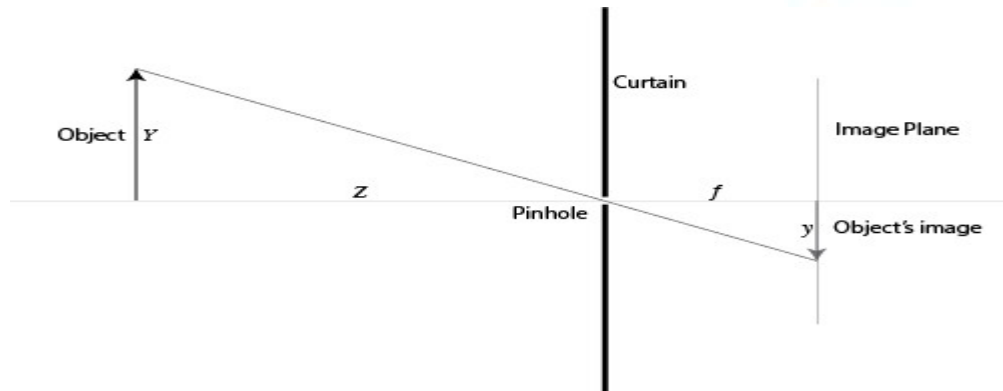
The Pinhole: Imagine a box with an opaque surface containing a minuscule hole.

Light Rays: Light from a 3D object travels in straight lines and enters through this pinhole.

Projection: Only the light rays that pass through the single point of the pinhole can reach the image plane (e.g., the film or sensor).

Inverted Image: Because the light rays cross at the pinhole, the image formed on the image plane is inverted.

Mathematical Relationship: The pinhole model provides a mathematical way to calculate where a 3D point in the world will be located on the 2D image plane.



Given

Y: Height of the object,

Z : Distance from the pinhole, and

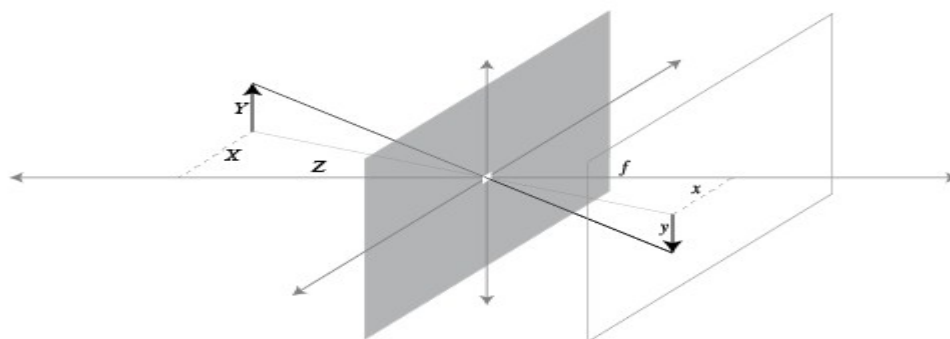
f : Distance between the pinhole and the image plane (also known as the focal length), we want to estimate

y : Height of the image of this object.

Using the similar triangles argument, we get: $Y/Z = y/f$.

$$y = fY/Z.$$

In 3D



Difference between Parallel and Perspective Projection in Computer Graphics

S.No	Parallel Projection	Perspective Projection
1	Parallel projection represents the object in a different way like telescope.	Perspective projection represents the object in three dimensional way.
2	In parallel projection, these effects are not created.	In perspective projection, objects that are far away appear smaller, and objects that are near appear bigger.
3	The distance of the object from the center of projection is infinite.	The distance of the object from the center of projection is finite.
4	Parallel projection can give the accurate view of object.	Perspective projection cannot give the accurate view of object.
5	The lines of parallel projection are parallel.	The lines of perspective projection are not parallel.
6	Projector in parallel projection is parallel.	Projector in perspective projection is not parallel.
7	Two types of parallel projection : 1.Orthographic 2.Oblique	Three types of perspective projection: 1.one point perspective 2.Two point perspective 3. Three point perspective
8	It does not form realistic view of object.	It forms a realistic view of object.

Camera calibration including intrinsic and extrinsic parameters

Camera calibration determines a camera's internal properties (intrinsic parameters like focal length and distortion coefficients) and its external position and orientation relative to the world (extrinsic parameters).

Intrinsic Parameters

These are fixed internal properties of the camera:

- **Focal Length (f):** Determines the field of view.
- **Principal Point (cx, cy):** The image center.
- **Distortion Coefficients (k1, k2, p1, p2, k3):** Account for radial and tangential distortions in the lens, which cause points to bend away from their true positions.

Extrinsic Parameters

These parameters describe the camera's pose in the 3D world:

- **Rotation Matrix (R):** A 3x3 matrix defining the camera's orientation.
- **Translation Vector (T):** A 3x1 vector defining the camera's position in the world.

How Calibration Works

- **Projective Transformation:** The overall process aims to find a projection matrix (P) that maps a 3D world point (X) to its corresponding 2D image point (x) using the equation $x = P * X$.
- **Known 3D-2D Correspondences:** You need a set of known 3D world points and their corresponding 2D image points. A common method involves imaging a patterned object, like a chessboard, and identifying the precise locations of its inner corners in the image.
- **Solving for Parameters:** A system of equations is solved to determine the intrinsic and extrinsic parameters by minimizing the error between the projected 3D points and the actual 2D image points.

Why Camera Calibration is Important

- **3D Reconstruction:** Allows for the reconstruction of 3D objects from 2D images.
- **Augmented Reality (AR):** Enables the correct overlay of virtual objects into the real world.
- **Image Undistortion:** Corrects the geometric distortions introduced by the camera lens, leading to more accurate measurements and visualizations.

S.No	Questions
01	Define Image segmentation? Explain types of Image Segmentation
02	Compare between Image Segmentation, Object Detection and Image Classification.
03	Define Descriptors and matching, explain process involved in image processing
04	With neat block diagram explain Scale-Invariant Feature Transform (SIFT) with python code.
05	Explain Oriented FAST and Rotated BRIEF (ORB) with python code.
06	Explain , Feature matching techniques.
07	Write short note on 1) Speeded-Up Robust Features (SURF) 2) Local Binary Pattern (LBP) 3) Histogram of Oriented Gradients (HOG)
08	Explain , Epipolar geometry in computer vision.
09	With neat block diagram Explain Working of Stereo Vision.
10	Explain Image formation and camera models.
11	List difference between Parallel and Perspective Projection in Computer Graphics
12	Write short note on Camera calibration

D Y PATIL
INTERNATIONAL
UNIVERSITY
AKURDI PUNE